Today's agenda

- Cherry Pickup
- Intro
- Types of graph
- Storage

AlgoPrep

Cherry Pickup

// Correct idea

5 steps → right
4 steps → down
4 9 steps

Ex:

	0	1	2	3	4	5
0	1	1	1	1	0	0
1	0	0	0	1	0	1
2	1	0	0	1	0	0
3	0	0	0	1	0	0
4	0	0	0	1	1	1

P1 P2 P1 P2 P2 P1 P2 dest 5×6 P1 →

↳ we will be doing 2 TL - BR instead of full cycle.

P1 (→ row1, → col1)	P2 (→ row2, → col2)	
R	R	→ y
D	D	→ x
R	D	→ w
D	R	→ z

max(x, y, w, z) + mat[i][j]

	0	1	2	3	4	5
0	1	1	1	1 P2	0	0
1	0	0	0	1	0	1
2	1	0 P1	0	1	0	0
3	0	0	0	1	0	0
4	0	0	0	1	1	1

row1 + col1 == row2 + col2

col2 = row1 + col1 - row2

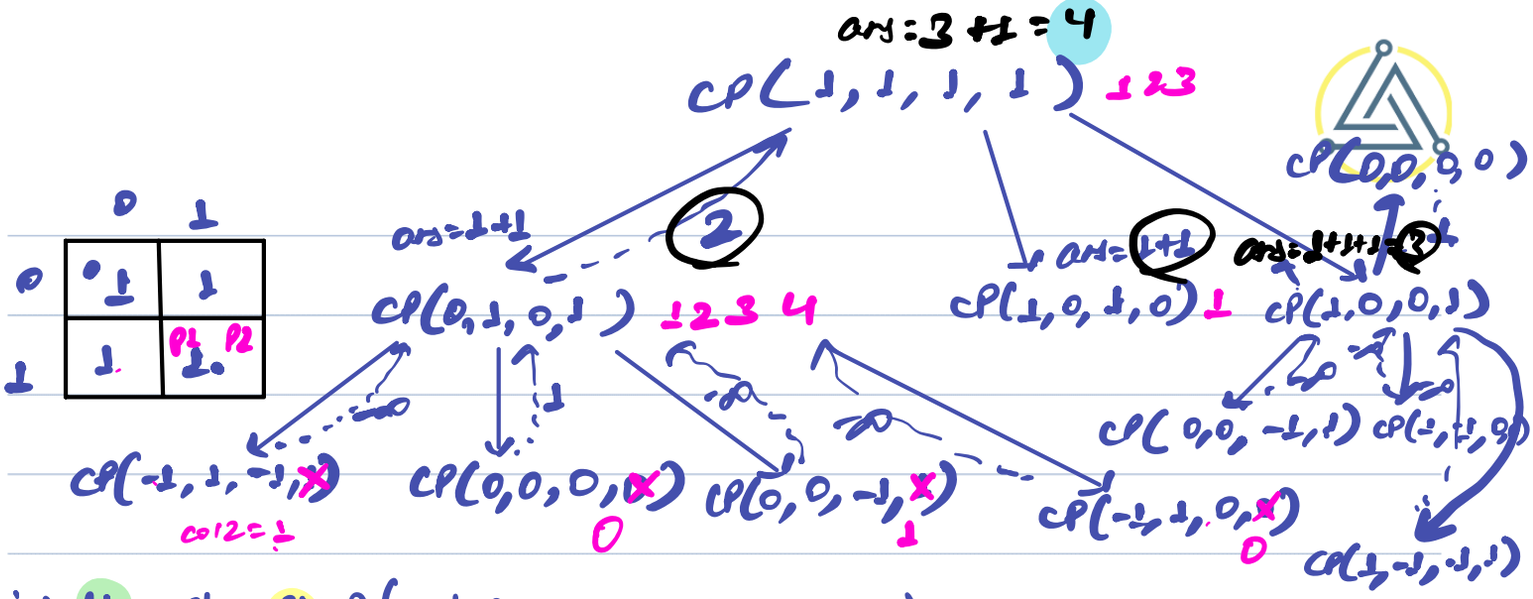
//Pseudo code

```
int dp[n][n][n][n] = {-1}

int cherryPickup (int[][] mat, int row1, int col1, int row2, int col2){
    int col2 = row1 + col1 - row2
    if (row1 < 0 || row2 < 0 || col1 < 0 || col2 < 0 || mat[row1][col1] == -1
        || mat[row2][col2] == -1){ return -∞; }
    if (row1 == 0 && col1 == 0 && row2 == 0 && col2 == 0){
        return mat[row1][col1];
    }
    if (dp[row1][col1][row2][col2] != -1){ return dp[row1][col1][row2][col2]; }
    int f1 = cherryPickup (mat, row1-1, col1, row2-1, col2);
    int f2 = cherryPickup (mat, row1, col1-1, row2, col2-1);
    int f3 = cherryPickup (mat, row1, col1-1, row2-1, col2);
    int f4 = cherryPickup (mat, row1-1, col1, row2, col2-1);

    int ans = max(f1, f2, f3, f4);
    int conti = 0;
    if (row1 == row2 && col1 == col2){
        conti = mat[row1][col1];
    }
    else{
        conti = mat[row1][col1] + mat[row2][col2];
    }

    dp[row1][col1][row2][col2] = ans + conti;
    return ans + conti;
}
```

T.C: $O(n^3)$

S.C: $O(n^3)$

AlgoPrep




```
1 int t1 = CherryPickup(mat, row1-1, col1, row2-1, col2);
2 int t2 = CherryPickup(mat, row1, col1-1, row2, col2-1);
3 int t3 = CherryPickup(mat, row1, col1-1, row2-1, col2);
4 int t4 = CherryPickup(mat, row1-1, col1, row2, col2-1);
```

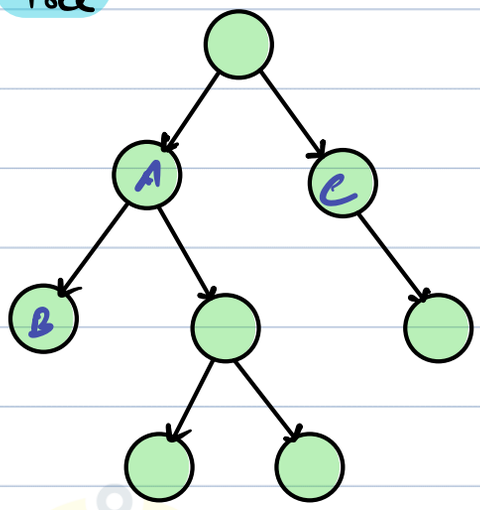
```
int ans = max(t1, t2, t3, t4);
    int conti=0;
    if(row1==row2 && col1==col2){
        conti= mat[row1][col1];
    }
    else{
        conti= mat[row1][col1]+mat[row2][col2];
    }
    return ans + conti;
```

AlgoPrep

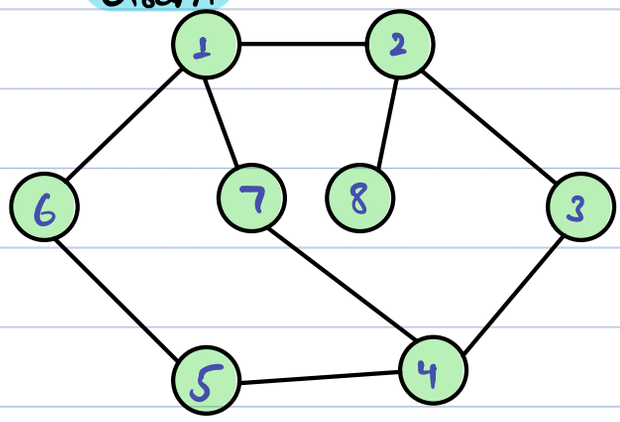
Intro

↳ Graph: Connection of nodes and edges

1. Tree



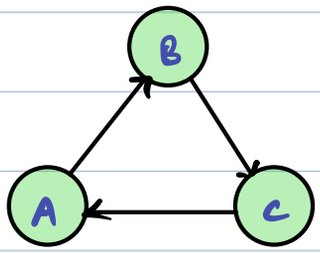
Graph

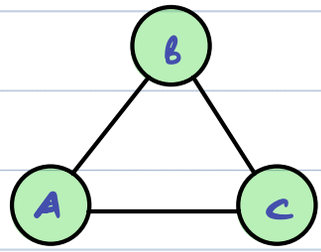


AlgoPrep

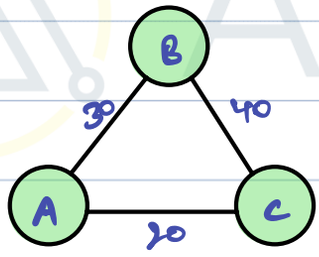
* Classification of graphs

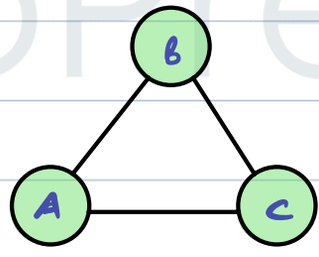
Case I: Based on types of edges.





Case II: Based on weight of edge

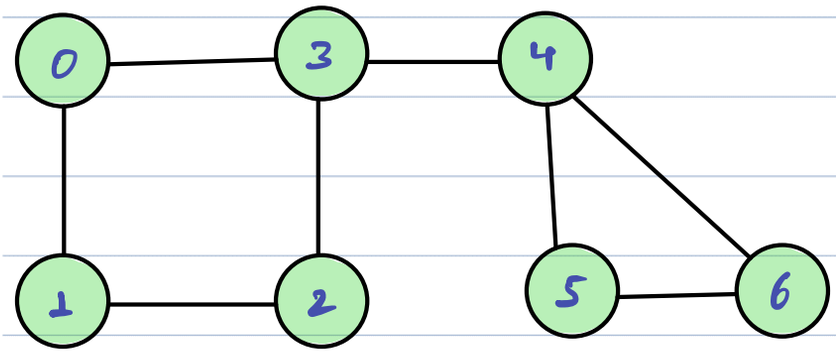




AlgoPrep

Storing a graph

undirected graph



input

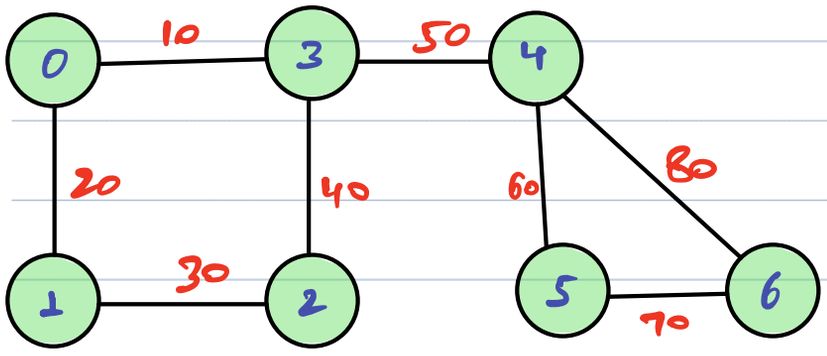
N = 7 → no. of nodes

M = 8 → no. of edges

3	4
1	2
2	3
4	6
0	1
4	5
5	6
0	3

AlgoPrep

② adjacency list representation



N=7 M=8

edges

3	4
1	2
2	3
4	6
0	1
4	5
5	6
0	3

AlgoPrep

AlgoPrep

AlgoPrep

Q) Find max no. of edges possible if N nodes are present in Graph.

AlgoPrep